

多层 FFT 海浪 + 潮汐 + Vulkan 水体渲染

阶段 5 起完整学习路线总结

目标：深入理解并手写实现「水体模拟物理 + Vulkan 实时渲染」

适用进度：阶段 4 已完成，阶段 5 Tessendorf 海洋谱 + FFT 刚开始

核心结论

不要直接把多层 FFT、潮汐和 iWave 全部塞进 Vulkan。先在纯 CPU 中验证物理数据，再接入 Vulkan 网格与 shader。

目录

1. 1. 总目标与系统边界
2. 2. 总体架构：FFT、潮汐、iWave、Vulkan 的分工
3. 3. 阅读与代码参照路线
4. 4. 六周执行计划
5. 5. 阶段 5A-5E：CPU 物理模块
6. 6. 阶段 6：Vulkan 接入
7. 7. 阶段 7-9：着色、iWave、GPU 化
8. 8. 验收清单与当前第一步
9. 9. 参考资料

1. 总目标与系统边界

当前目标已经从“单层 Tessendorf FFT 海面”升级为“多层 FFT 远洋海浪 + 潮汐水位变化 + 可扩展 iWave 近场交互 + Vulkan 实时渲染”。

最终目标系统

= 多层 FFT 背景海浪

+ 潮汐水位 / 慢速潮流

+ 近场交互波 (后续 iWave 或 2D wave equation)

+ Vulkan 网格、位移贴图、法线贴图和水体着色

本阶段最重要的边界

现在要做	现在不要做
CPU 单层 FFT、输出 height/displacement/normal debug 图	直接写复杂水体 shader
多层 FFT 的物理合成和参数组织	一上来实现 GPU FFT / compute shader
独立 TideSystem，慢速改变全局水位	把潮汐强行塞进 Phillips spectrum
为 Vulkan 设计清晰的数据接口	直接复制 iWaveOcean 源码

2. 总体架构：FFT、潮汐、iWave、Vulkan 的分工

OceanSystem

|

| -- MultiLayerFFT0cean // 大范围背景海浪

| | -- FFTLayer_0 // 长波 / swell / 大尺度起伏

| | -- FFTLayer_1 // 中尺度风浪

| | -- FFTLayer_2 // 小尺度 ripple / normal detail

|

| -- TideSystem // 全局水位 + 慢速潮流

|

| -- LocalInteractionSystem // 后续: iWave / 2D wave equation / wake

|

| -- WaterMeshSystem // Vulkan 网格、LOD、数据上传

|

| -- WaterShadingSystem // Fresnel、反射、折射、泡沫、水深颜色

一句话分工

模块	解决的问题	数据特征
Multi-layer FFT	大海本身如何起伏	秒级、高频、周期性 patch、频域生成
TideSystem	海平面如何慢慢涨落	分钟/小时级、低频、全局水位
iWave / local waves	物体如何扰动水面	局部、可交互、支持源项和障碍反射
Vulkan renderer	如何把数据画出来	网格、贴图、descriptor、shader、同步

3. 阅读与代码参照路线

阅读顺序不要平均用力。先读会直接决定代码结构的材料，工业案例和插件横评后置。

第一组：现在必须精读

材料	阅读目的	对应实现
从零手写水体渲染器 · 学习方案.pdf	明确阶段 5/6 的边界、验收和 Vulkan 接入顺序	stage5_ocean_cpu、WaterSurfaceMesh
simulating-ocean-water.pdf	理解 Tessendorf FFT、Phillips 谱、色散关系、 $h_0(k)$ 、 $h(k,t)$	FFTLayer
interactive-water-surfaces.pdf	理解 iWave 为什么补足 FFT 的交互短板	后续 LocalInteractionSystem
sdao/iWaveOcean	只看结构与公式映射，不复制 GPL 代码	Ambient 对应 FFT；Dynamics 对应 iWave

第二组：辅助理解

- Fast Water Simulation Using Height Fields：理解高度场、水坑/池塘/交互波。
- 波动方程书籍章节：理解线性叠加、Cauchy 问题、能量守恒。
- 基于二维波动方程的水波效果：理解局部交互波的工程化做法。
- Wave Particles：理解船波、障碍反射和局部扰动的另一类方案。

第三组：后面再读

- Sea of Thieves、Horizon Forbidden West：工业系统、艺术控制、breaking waves、baked simulation。
- Underwater optical effects：阶段 7 之后再读，用于水下光学、caustics、god rays。
- 插件横评、浮力文档、GDC VFX：系统设计、泡沫、岸线、浮力和工具化参考。

4. 六周执行计划

周次	主题	产出	进入下一阶段条件
第 1 周	单层 Tessendorf FFT	height_t000/030/060.ppm ； 单层 FFTLayer	无 NaN/Inf；风向和风速对结果有效
第 2 周	多层 FFT	layer0/1/2 debug 图； combined displacement/normal	能解释为什么不同 L、N、seed、amplitude 分层
第 3 周	潮汐 + Vulkan 水面网格	TideSystem；sloped beach debug；静态/动态水面网格	水位可控；网格可起伏；validation 零 error
第 4 周	基础水体 shading	Fresnel、天空反射、太阳高光、水深颜色、normal detail	几何和法线稳定，画面无严重闪烁
第 5 周	CPU iWave 近场交互	source/obstruction/反射的 128×128 iWave demo	鼠标/物体扰动可传播并与障碍反射
第 6 周	iWave 接入 Vulkan + 整理	local height 合成；阶段验收报告	远场 FFT + 潮汐 + 局部扰动可同时工作

5. 阶段 5A-5E：CPU 物理模块

5A. 单层 Tessendorf FFT

先实现最小可验证的单层 FFT：N、L、风速、风向、amplitude、damping、time 输入，输出 height(x,z,t)。

```
FFTLayer 数据流
k-grid
-> Phillips spectrum P_h(k)
-> Gaussian random h0(k)
-> time evolution h(k,t)
-> IFFT
-> height field
```

5B. 位移、斜率、法线、Jacobian

height 只能让网格上下动。真实海面渲染还需要横向位移、斜率和法线。建议在频域中直接生成 slope 和 displacement，再 IFFT 回空间域。

频域量	空间域用途
$h(k,t)$	height / Dy
$i*k_x*h(k,t)$	slopeX, 用于 normal
$i*k_z*h(k,t)$	slopeZ, 用于 normal
$-i*k_x/ k * h(k,t)$	Dx, choppy 横向位移
$-i*k_z/ k * h(k,t)$	Dz, choppy 纵向位移

5C. 多层 FFT

层	用途	建议参数倾向
Layer 0: Swell	远处长波、大尺度海面轮廓	L=1000-2000m; amplitude 较大; choppy 中等
Layer 1: Wind waves	中尺度风浪、主要几何起伏	L=150-400m; 风向可略偏; choppy 较强
Layer 2: Ripples	小波纹和 normal detail	L=20-80m; 位移弱, 主要贡献法线

多层合成时，normal 不建议直接相加 normal；更稳妥的是先累加 slope，再重新 normalize。

```
slopeX_total = sum(layer.slopeX * layer.weight)
slopeZ_total = sum(layer.slopeZ * layer.weight)
normal = normalize(vec3(-slopeX_total, 1.0, -slopeZ_total))
```

5D. TideSystem

潮汐是全局低频水位变化，不属于 FFT 频谱。第一版用正弦函数模拟水位，用余弦函数模拟慢速潮流即可。

```
tideHeight(t) = meanSeaLevel + A * sin(2*pi*t / period + phase)
tideCurrent(t) = currentDirection * C * cos(2*pi*t / period + phase)
waterY = tideHeight(t) + sum(fftHeight_i)
```

5E. OceanSurfaceState 合成器

```
final displacement = vec3(0, tideHeight, 0)
    + sum(FFTLayer_i.displacement)
    + optional localWaveDisplacement

final normal = normalize(vec3(-sumSlopeX, 1, -sumSlopeZ))
```

6. 阶段 6: Vulkan 接入

只有在 CPU 多层 FFT + TideSystem 通过验收后，才回到 Vulkan。第一版建议 CPU 合成一张 displacement texture 和一张 normal texture，降低 shader 和 descriptor 复杂度。

```
CPU MultiLayerFFTOcean::Compute(time)
-> combinedDisplacement[N*N]
-> combinedNormal[N*N]
-> staging buffer
-> VkImage displacementTexture / normalTexture
-> descriptor set
-> vertex shader sample displacement
-> fragment shader sample normal
```

方案	优点	缺点	推荐阶段
A. CPU 合成后上传一张最终 texture	最简单；shader 轻；适合学习	每帧上传较多；GPU 灵活度低	先做
B. 每层单独上传，shader 中合成	层权重/LOD/距离衰减灵活	descriptor 和采样复杂	稳定后再做
C. GPU compute FFT	性能上限最高；避免 CPU 上传	实现难度最高	最后优化

Vertex shader 最小接入

```
vec3 D = texture(displacementMap, uv).xyz;
worldPos.xyz += D;
worldPos.y += tideHeight; // 如果 tide 没有预先合入 texture
```

7. 阶段 7-9: 着色、iWave、GPU 化

阶段 7: 水体 shading

- 先做 Fresnel + sky reflection + sun highlight。
- 再做 depth-based absorption：水深越深颜色越深、透明度越低。
- 最后做 foam：第一版可基于 Jacobian、坡度、水深和岸线距离。

阶段 8：iWave 近场交互

iWave 不替代 FFT，而是补充 FFT 不擅长的“物体交互、障碍反射、局部扰动”。先 CPU 实现 128×128 demo，再合成到 Vulkan 水面。

```
height_total = tideHeight
               + multiLayerFFTHeight
               + localIWaveHeight
```

阶段 9：性能优化

- CPU FFTW 版本稳定后，再考虑 GPU compute FFT。
- 大浪层低频可降低更新率；小波层可只贡献 normal。
- 近处高密度网格，远处低密度网格；后续再考虑 CDLOD / clipmap。

8. 验收清单与当前第一步

CPU 物理验收

- 单层 FFT：height 连续变化，不随机闪烁。
- 多层 FFT：layer0/1/2 分别可视化，combined 图无棋盘格和爆点。
- 法线：长度接近 1；slope 方向和波形一致。
- 潮汐：水位慢速周期变化，不影响 FFT 的高频波形。
- 全部输出无 NaN、无 Inf，固定 seed 可复现。

Vulkan 验收

- 水面网格可用 index buffer 绘制，线框和实体都正确。
- displacement texture 每帧更新后，网格起伏稳定。
- resize / minimize / restore 不崩；validation layer 零 error。
- tideHeight 可通过 UBO 控制整体水位。

你现在马上做的第一步

```
新建 experiments/multilayer_fft_cpu
只实现：
1. FFTLayer
2. MultiLayerFFT0cean
3. TideSystem
4. 输出 combined_height.ppm / combined_normal.ppm

暂时不碰 Vulkan。
```

9. 参考资料

来源	用于什么
从零手写水体渲染器 · 学习方案.pdf	阶段划分、阶段 5/6 验收、从 CPU 物理到 Vulkan 的接入顺序
Jerry Tessendorf, Simulating Ocean Water	FFT 海洋、色散关系、Phillips 谱、choppy wave、水体光学基础
Jerry Tessendorf, Interactive Water Surfaces	iWave：二维卷积、源项、mask、障碍反射、局部交互波
sdao/iWaveOcean GitHub repository	工程结构参照：Ambient=FFT 背景浪；Dynamics=iWave 动态浪；GPL-3.0，仅作参考学习参照
Fast Water Simulation Using Height Fields / Wave Particles / Horizon / Sea of Thieves 等材料	扩展理解：局部交互、工业实践、泡沫、水下和性能优化
最终记忆句 FFT 解决“大海本身怎么动”；潮汐解决“海平面怎么慢慢涨落”；iWave 解决“物体怎么扰动水”；Vulkan 解决“这些数据怎么画出来”。	